

Homework 3

Problem 6.1

1.

```
In [ ]: from rectified_flow.datasets.toy_gmm import TwoPointGMM

n_samples = 50000
pi_0 = TwoPointGMM(x=0.0, y=7.5, std=0.5, device=device)
pi_1 = TwoPointGMM(x=15.0, y=7.5, std=0.5, device=device)
D0 = pi_0.sample([n_samples])
D1, labels = pi_1.sample_with_labels([n_samples])
labels.tolist()

plt.figure(figsize=(3, 3))
plt.title(r'Samples from $\pi_0$ and $\pi_1$')
plt.scatter(D0[:, 0].cpu(), D0[:, 1].cpu(), alpha=0.5, label=r'$\pi_0$')
plt.scatter(D1[:, 0].cpu(), D1[:, 1].cpu(), alpha=0.5, label=r'$\pi_1$')
plt.legend()
```

```
In [ ]: optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
batch_size = 1024

losses = []

for step in range(5000):
    optimizer.zero_grad()
    idx = torch.randperm(n_samples)[:batch_size]
    x_0 = D0[idx].to(device)
    x_1 = D1[idx].to(device)

    loss = rectified_flow.get_loss(x_0, x_1)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

    if step % 1000 == 0:
        print(f"Epoch {step}, Loss: {loss.item()}")

plt.plot(losses)
```

```
In [ ]: from rectified_flow.samplers import EulerSampler
from rectified_flow.utils import visualize_2d_trajectories_plotly

euler_sampler_1rf_unconditional = EulerSampler(
    rectified_flow=rectified_flow,
```

```

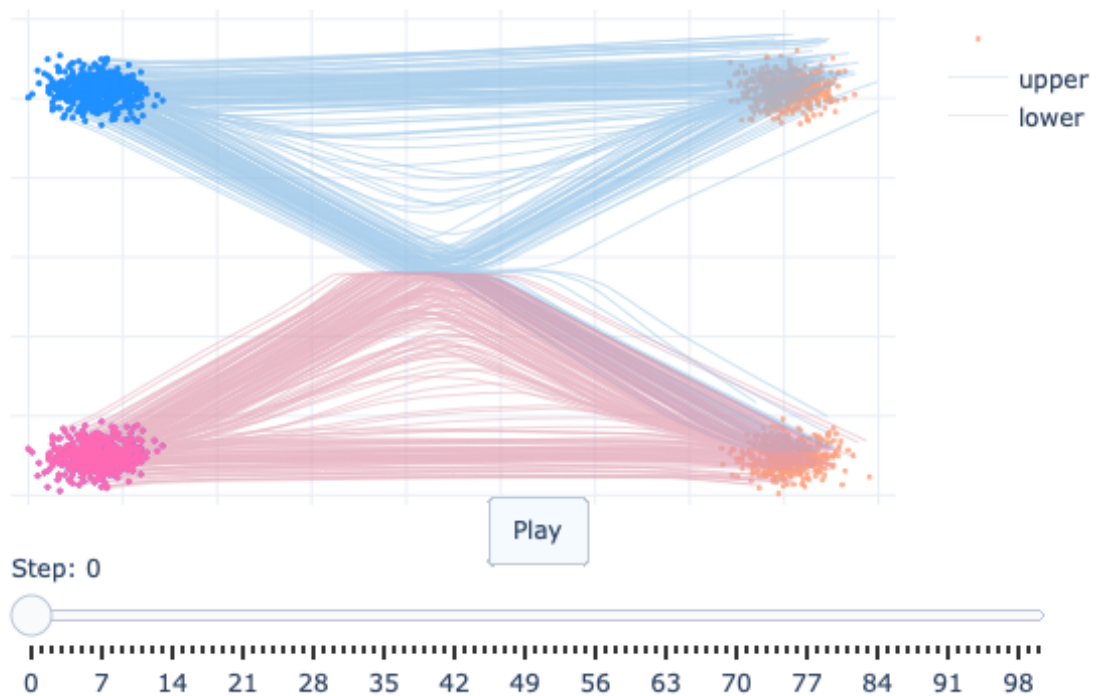
num_steps=100,
)

traj_upper = euler_sampler_1rf_unconditional.sample_loop(x_0=x_0_upper
traj_lower = euler_sampler_1rf_unconditional.sample_loop(x_0=x_0_lower

visualize_2d_trajectories_plotly(
    trajectories_dict={"upper": traj_upper, "lower": traj_lower},
    D1_gt_samples=D1[:1000],
    num_trajectories=200,
    title="Unconditional 1-Rectified Flow",
)

```

Unconditional 1-Rectified Flow



2.

```

In [ ]: from torch.utils.data import TensorDataset, DataLoader

def generate_data(batch_size: int = 200, device: str = "cpu") -> torch
    """
    Generate synthetic 2D datasets without rewards.

    Parameters
    -----
    data : {"rings", "8gaussians", "2spirals", "checkerboard"}
    batch_size : int
    device : str

    Returns

```

```

-----
X : torch.FloatTensor of shape (batch_size, 2)
"""
def torch_linspace_exclusive(start, stop, steps, device="cpu"):
    return torch.linspace(start, stop, steps + 1, device=device)[:
        steps]

half = batch_size // 2
n = torch.sqrt(torch.rand(half, 1, device=device)) * (3 * np.pi)

d1x = -torch.cos(n) * n + torch.rand(half, 1, device=device) * 0.5
d1y = torch.sin(n) * n + torch.rand(half, 1, device=device) * 0.5
spiral1 = torch.cat([d1x, d1y], dim=1)

spiral2 = -spiral1
X = torch.cat([spiral1, spiral2], dim=0) / 3.0
X = X + torch.randn_like(X) * 0.1

perm = torch.randperm(X.size(0), device=device)
# if batch_size is odd, drop the last extra sample after permuting
return X[perm][:batch_size].float()

```

```

In [ ]: batch_size = 1024
dataset_size = 50000
D1 = generate_data(batch_size=dataset_size, device="cpu")

plt.figure(figsize=(6, 6))

plt.scatter(
    D0[:, 0].cpu(),
    D0[:, 1].cpu(),
    s=4,
    alpha=0.25,
    label=r"$\pi_0$"
)

plt.scatter(
    D1[:, 0].cpu(),
    D1[:, 1].cpu(),
    s=4,
    alpha=0.5,
    label=r"$\pi_1$"
)

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Source and Rings Target Distributions")
plt.axis("equal")
plt.legend()
plt.show()

```

```

In [ ]: model = MLPVelocity(2, hidden_sizes = [128, 128, 128]).to(device)

```

```
rectified_flow = RectifiedFlow(
    data_shape=(2,),
    velocity_field=model,
    interp="straight",
    source_distribution=pi_0,
    device=device,
)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
batch_size = 1024

losses = []

for step in range(5000):
    optimizer.zero_grad()
    idx = torch.randperm(n_samples)[:batch_size]
    x_0 = D0[idx].to(device)
    x_1 = D1[idx].to(device)

    loss = rectified_flow.get_loss(x_0, x_1)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

    if step % 1000 == 0:
        print(f"Epoch {step}, Loss: {loss.item()}")

plt.plot(losses)

euler_sampler = EulerSampler(
    rectified_flow=rectified_flow,
    num_steps=100,
)

traj_upper = euler_sampler.sample_loop(x_0=x_0_upper).trajectories
traj_lower = euler_sampler.sample_loop(x_0=x_0_lower).trajectories

visualize_2d_trajectories_plotly(
    {"1rf 2-Spirals": euler_sampler.trajectories},
    D1[:1000],
    num_trajectories=200,
    title="2-Spirals Rectified Flow",
)
```

2-Spirals Rectified Flow



3. Below is my implementation change for the `get_loss` function:

```
In [ ]: def get_loss(
    self,
    x_0: torch.Tensor | None,
    x_1: torch.Tensor,
    t: torch.Tensor | None = None,
    **kwargs,
):
    """Compute the loss of the rectified flow model, given samples

    This method calculates the loss by interpolating between the g
    computing the velocity field at the interpolated points, and c
    The result is weighted by the specified time weight class and

    Args:
        x_0 (`torch.Tensor` or `None`):
            Samples from the source distribution `pi_0`, with shap
            where `B` is the batch size and `D_1, D_2, ..., D_n` a
            If `None`, samples are drawn from the source distribut
        x_1 (`torch.Tensor`):
            Samples from the target distribution `pi_1`, with the
        t (`torch.Tensor` or `None`, *optional*, defaults to `None`):
            A tensor of time steps, with shape `(B,)`, where each
            If `None` or not provided, training times are sampled
        **kwargs:
            Additional keyword arguments passed to the velocity fi
```

```

Returns:
    loss (`torch.Tensor`):
        A scalar tensor representing the computed loss value.
    """
    if x_0.shape != x_1.shape:
        raise ValueError(
            f"x_0 and x_1 must have the same shape, but got {x_0.s
        )

    # Sample t
    t = self.sample_train_time(x_1.shape[0]) if t is None else t

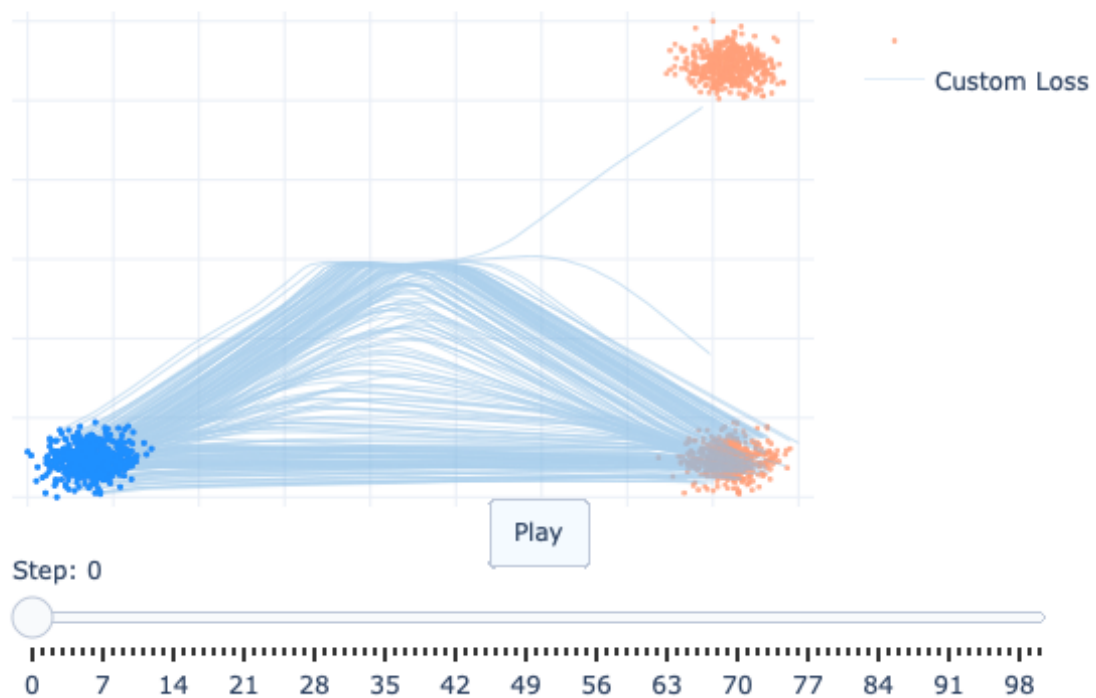
    x_t = t * x_1 + (1.0 - t) * x_0

    sample_loss = torch.mean((x_1 - x_0) - self.get_velocity(x_t,
    return torch.mean(sample_loss)

```

This was the output:

Unconditional 1-Rectified Flow Custom Loss



4.

```

In [ ]: experiments = [
    {
        "name": "Baseline",
        "lr": 1e-3,
        "batch_size": 1024,

```

```

        "hidden_sizes": [128, 128, 128],
        "steps": 5000
    },
    {
        "name": "Lower learning rate",
        "lr": 1e-4,
        "batch_size": 1024,
        "hidden_sizes": [128, 128, 128],
        "steps": 5000
    },
    {
        "name": "Wider model",
        "lr": 1e-3,
        "batch_size": 1024,
        "hidden_sizes": [256, 256, 256],
        "steps": 5000
    },
    {
        "name": "Smaller batch",
        "lr": 1e-3,
        "batch_size": 256,
        "hidden_sizes": [128, 128, 128],
        "steps": 5000
    }
]

for exp in experiments:
    print(f"Running experiment: {exp['name']}")
    model = MLPVelocity(2, hidden_sizes=exp["hidden_sizes"]).to(device)

    rectified_flow = RectifiedFlow(
        data_shape=(2,),
        velocity_field=model,
        interp="straight",
        source_distribution=pi_0,
        device=device,
    )

    optimizer = torch.optim.Adam(model.parameters(), lr=exp["lr"])
    batch_size = exp["batch_size"]
    steps = exp["steps"]

    losses = []

    for step in range(steps):
        optimizer.zero_grad()
        idx = torch.randperm(n_samples)[:batch_size]
        x_0 = D0[idx].to(device)
        x_1 = D1[idx].to(device)

        loss = rectified_flow.get_loss(x_0, x_1)
        loss.backward()

```

```

optimizer.step()
losses.append(loss.item())

if step % 1000 == 0:
    print(f"Epoch {step}, Loss: {loss.item()}")

plt.plot(losses)
plt.title(f"Loss Curve for {exp['name']}")
plt.xlabel("Steps")
plt.ylabel("Loss")
plt.show()

euler_sampler = EulerSampler(
    rectified_flow=rectified_flow,
    num_steps=100,
)

traj_upper = euler_sampler.sample_loop(x_0=x_0_upper).trajectories
traj_lower = euler_sampler.sample_loop(x_0=x_0_lower).trajectories

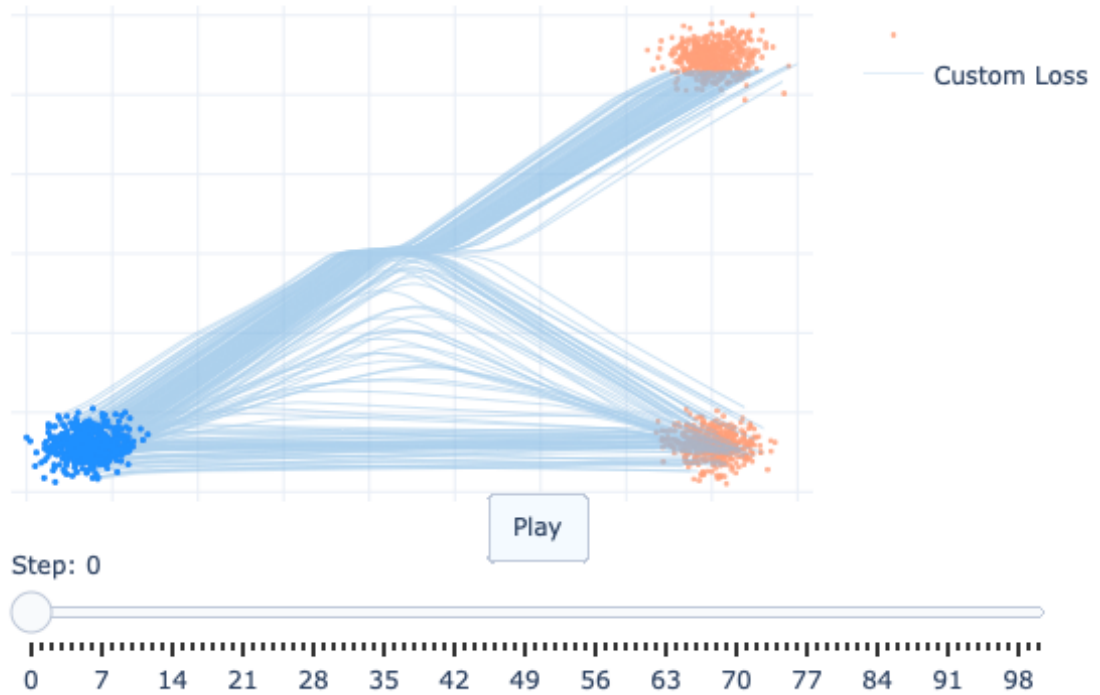
visualize_2d_trajectories_plotly(
    {"Custom Loss": euler_sampler.trajectories},
    D1[:1000],
    num_trajectories=200,
    title=f"Trajectories for {exp['name']}",
)

```

For each experiment, I got the following results:

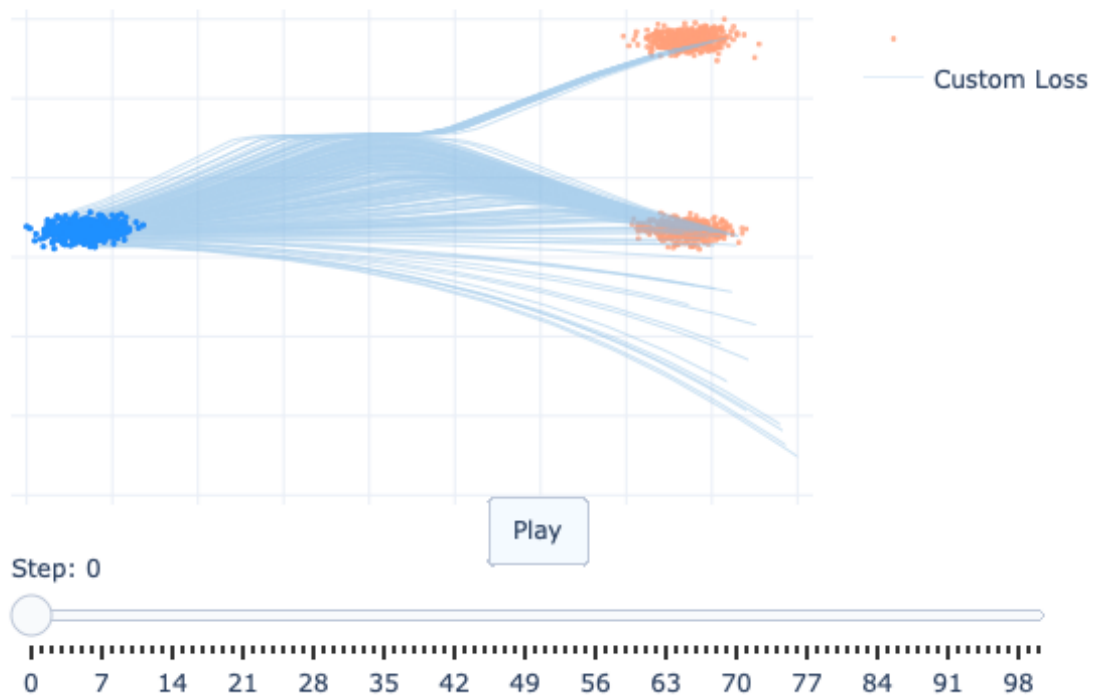
Baseline experiment:

Trajectories for Baseline



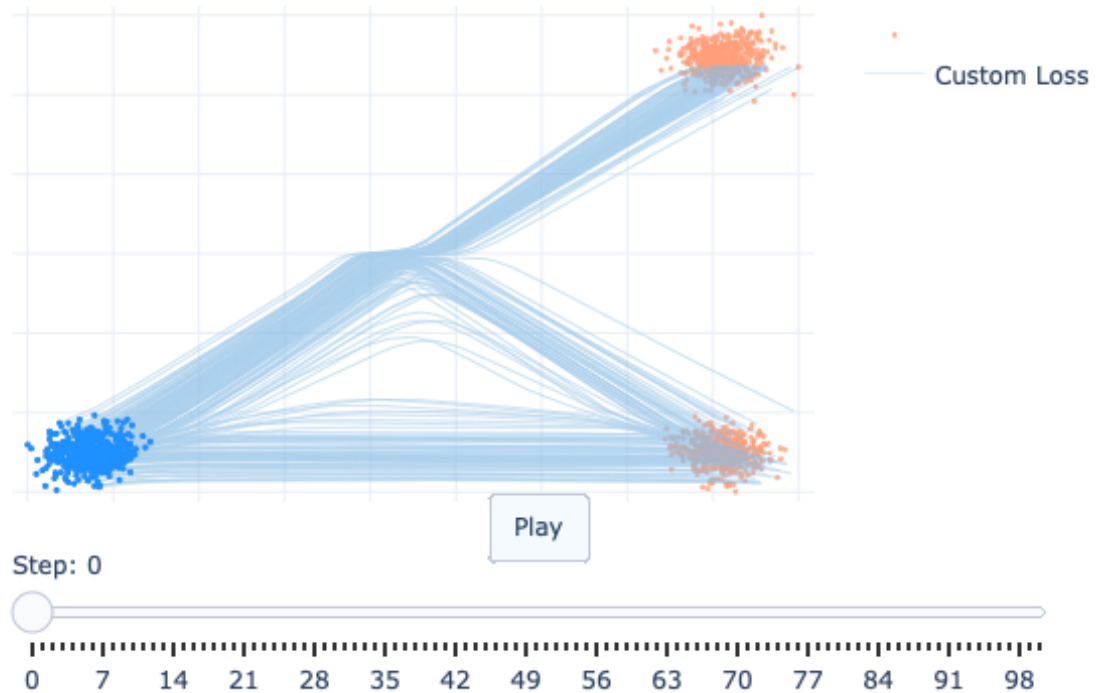
Low LR:

Trajectories for Lower learning rate



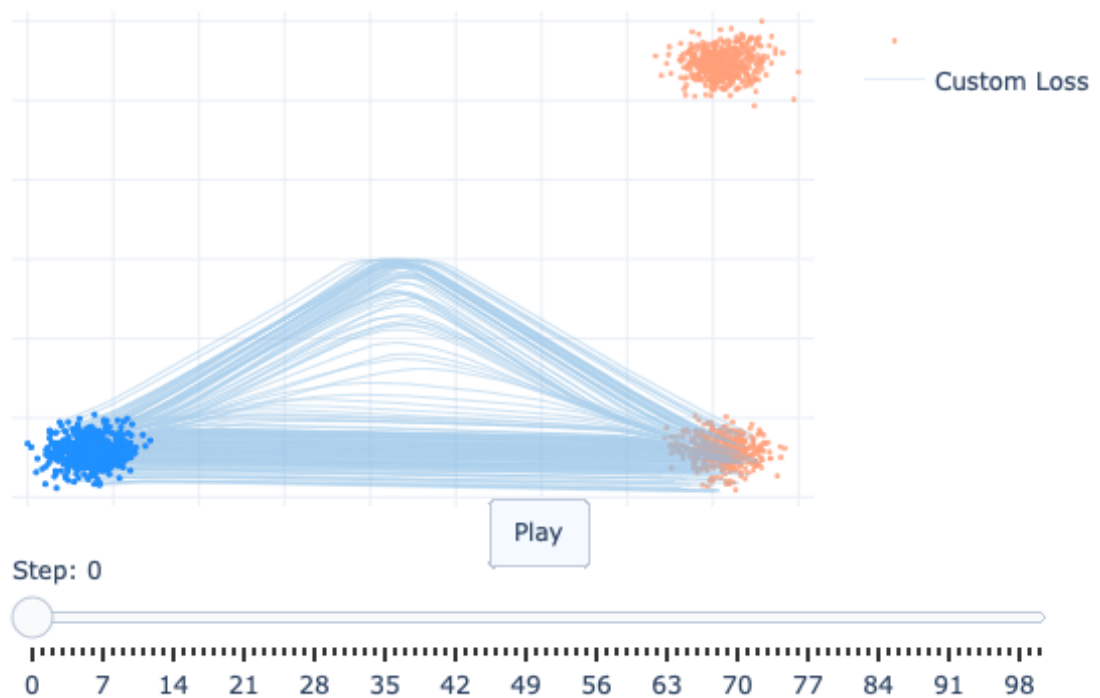
Wider:

Trajectories for Wider model



Smaller Batch:

Trajectories for Smaller batch



The bealine configuration produced more stable training and coverage for the 2 data patches. Reducing the learning rate caused convergence to be slower and leading to more errors when producing the trajectories. The wider model had a

similar performance to the baseline model; however, it take much more compute power and still produced minor errors compared to the baseline model. The smaller batch did not traject towards one of the data patches and only produced towards the lower trajectory.

Problem 6.2

```
In [ ]: x_0 = torch.randn(100, *data_shape).to(device)

x_t_latest = x_0.clone()
x_t_early = x_0.clone()

N = 100

earlier_model = flow_model.__class__.from_pretrained(
    "./checkpoints/flow_mnist-20"
).to(device)

# Implement Euler Sampler
with torch.inference_mode():
    for i in range(N):
        t = i / N
        t_b = t * torch.ones(x_0.shape[0], device=x_0.device, dtype=x_0.dtype)
        v_pred_latest = flow_model(x_t_latest, t_b)
        x_t_latest = x_t_latest + v_pred_latest * (1. / N)

        v_pred_early = earlier_model(x_t_early, t_b)
        x_t_early = x_t_early + v_pred_early * (1. / N)

print("Latest checkpoint")
plot_cifar_results(x_t_latest)

print("Early Checkpoint")
plot_cifar_results(x_t_early)
```

The above output shows the latest checkpoint image generation vs the earlier checkpoint. We can see that the earlier checkpoint is able to generate some numbers while still struggling on others such as 8 or 9. The latest checkpoint shows an image with clear numbers, indicating much better quality. The earlier checkpoint is from epoch 20 and the later checkpoint is from epoch 100 proving that later epochs have much better image quality.

```
In [ ]: from rectified_flow.samplers import EulerSampler, SDESampler

ema_flow.apply_shadow()
model_inference = flow_model.eval()
```

```

rf_inference = RectifiedFlow(
    data_shape=data_shape,
    velocity_field=model_inference,
    device=device,
)

euler_sampler_low = EulerSampler(rf_inference, num_steps=5)
sde_sampler_low = SDESampler(rf_inference, num_steps=5, noise_scale=5,

euler_sampler_high = EulerSampler(rf_inference, num_steps=75)
sde_sampler_high = SDESampler(rf_inference, num_steps=75, noise_scale=

```

```

In [ ]: print("*"*80)
        print("low num_step")
        print("*"*80)
        plot_cifar_results(x_1_euler_low)
        plot_cifar_results(x_1_sde_low)

        print("*"*80)
        print("high num_step")
        print("*"*80)
        plot_cifar_results(x_1_euler_high)
        plot_cifar_results(x_1_sde_high)

```

For the low num_step, I used a num_step of 5 and a num_step of 75. The low num_step definitely struggled a lot. There are quite a few numbers that failed to have good quality that has a clear representation of which number it is. Whereas other numbers it was highly successful in such as the number 1. For the high num_step, there are much more high quality numbers that are generated. There are still some small errors that occur where the number is not recognizable, but overall it has better output than the low num_step output

```

In [ ]: from rectified_flow.utils import plot_cifar_results

        print("Results for zero s")
        plot_cifar_results(x_1_euler_zero)
        plot_cifar_results(x_1_sde_zero)

        print("Results for 1.0 s")
        plot_cifar_results(x_1_euler_one)
        plot_cifar_results(x_1_sde_one)

        print("Results for 3.5 s")
        plot_cifar_results(x_1_euler_three)
        plot_cifar_results(x_1_sde_three)

        print("Results for 7.5 s")
        plot_cifar_results(x_1_euler_seven)
        plot_cifar_results(x_1_sde_seven)

```

With the use of $s=0$, we can definitely see the varying ways the numbers were generated. There were many numbers that were not generated properly and did not have visual quality. With $s=1$, the numbers started to improve from $s=0$. However, there is still a bunch of numbers being generated with no visual quality and varying ways to be represented. Some numbers were still being generated incorrectly. With $s=3.5$, we can see the numbers are being improved significantly in terms of quality. There still exist a number or so that was not generated properly, but the numbers were much more consistent visually. With $s=7.5$, there isn't much improvement here. We can see that it starts to mess up some numbers again and there starts to become numbers that vary visually. The number visual representation start to diverge at this point.

Problem 6.4

1. Let's have an X_0 as our starting point. Here we can derive the interpolation with respect to X_t .

$$\frac{dX_t}{dt} = -X_0 + a = a - X_0$$

This tells us the velocity of along the path. At any time t , suppose the current state is $X_t = x$. Then we can solve for X_0 .

$$\begin{aligned} x &= (1 - t)X_0 + ta \\ x - ta &= (1 - t)X_0 \\ \frac{x - ta}{1 - t} &= X_0 \end{aligned}$$

Which this is valid for any $t < 1$. We can then do substitution

$$\begin{aligned} v(x, t) &= a - \frac{x - ta}{1 - t} \\ v(x, t) &= \frac{a(1 - t) - x + ta}{1 - t} \\ v(x, t) &= \frac{a - x}{1 - t} \end{aligned}$$

Now, we can substitute x with $(1 - t)X_0 + ta$ which will give us the following

$$\begin{aligned}
 v(x, t) &= \frac{a - ((1 - t)X_0 + ta)}{1 - t} \\
 v(x, t) &= \frac{(1 - t)a - (1 - t)X_0}{1 - t} \\
 v(x, t) &= \frac{(1 - t)(a - X_0)}{1 - t} \\
 &= a - X_0
 \end{aligned}$$

2. Assume that $x_k = X_{t_k} = (1 - t_k)X_0 + t_k a$. We can apply the Euler update with this

$$\begin{aligned}
 x_{k+1} &= (1 - t_k)X_0 + t_k a + (t_{k+1} - t_k)(a - X_0) \\
 &= (1 - t_k)X_0 + t_k a + (t_{k+1} - t_k)a - (t_{k+1} - t_k)X_0 \\
 &= X_0(1 - t_k - t_{k+1} + t_k) + t_k a + (t_{k+1} - t_k)a \\
 &= X_0(1 - t_{k+1}) + t_k a + (t_{k+1} - t_k)a \\
 &= X_0(1 - t_{k+1}) + a(t_{k+1})
 \end{aligned}$$

This is exactly the straight interpolation. There is no Euler discretization error due to the velocity remaining constant for the entire trajectory which was proven in part 1. Thus, there is no error due to every path being perfectly straight with constant velocity.

Problem 6.5

1. Let $(X_0, X_1) = (0, 1)$ and $(X'_0, X'_1) = (1, 0)$ and $t = \frac{1}{2}$. We can solve the straight interpolation for (X_0, X_1) as follows

$$\begin{aligned}
 X_t &= (1 - \frac{1}{2})X_0 + (\frac{1}{2})X_1 \\
 &= (1 - \frac{1}{2})0 + \frac{1}{2}(1) \\
 &= \frac{1}{2}
 \end{aligned}$$

Then we can solve for (X'_0, X'_1)

$$\begin{aligned}
 X'_t &= (1 - \frac{1}{2})X'_0 + (\frac{1}{2})X'_1 \\
 &= (1 - \frac{1}{2})1 + \frac{1}{2}(0) \\
 &= 1 - \frac{1}{2} \\
 &= \frac{1}{2}
 \end{aligned}$$

This example proves that there exist 2 pairs where they cross paths at the same time.

2. A function must return one output per input. Allowing two different velocities for one input would mean that the future trajectory is not uniquely determined. This means the same starting location at the same time could move in a non-deterministic manner. This contradicts the deterministic ODE requirements that states that the present location and time must uniquely determine the direction of motion.
3. The equation can be broken down into steps. First, we would need to consider all endpoint pairs that pass through x at time t . For each pair, we must then find the corresponding velocity. Then, average the velocities according to the probabilities. This is the steps to solve for this equation hence it assigns a single velocity by averaging all the directions at location x at time t .

Problem 6.8

1. Multiplying an objective by a positive constant changes the numerical value but not the location of the minimum. Therefore minimizing the weighted objective can adhere to the conditional mean property. Therefore, for all time where $w(t) > 0$ we would be minimizing $v_t^*(x) = \mathbb{E}[X_1 - X_0 | X_t = t]$.
2. $w(t)$ matters for the single neural network $v^\theta(x, t)$ because the choice of $w(t)$ for all time values must learn the velocity field at every stage of the flow. Unlike the unrestricted case, where each t can have its own perfect function, a real neural network has limited capacity and may not fit all time regions equally well. $w(t)$ controls how strongly errors at each time contribute to the total training loss. If $w(t)$ is large for a certain range of t , mistakes in that region produce larger loss values and larger gradient updates, so the optimizer prioritizes improving the model there.

Problem 6.9

1. We know that X_1 and X_t and X_0 and X_t are jointly gaussian so we can use conditional expectation. Therefore we must compute the following

$$\mathbb{E}[X_1 | X_t = x] = \mathbb{E}[X_1] + \frac{\text{Cov}(X_1, X_t)}{\text{Cov}(X_t)}(x - \mathbb{E}[X_t])$$

$$\mathbb{E}[X_0|X_t = x] = \mathbb{E}[X_0] + \frac{Cov(X_0, X_t)}{Cov(X_t)}(x - \mathbb{E}[X_t])$$

First lets solve for $\mathbb{E}[X_t]$.

$$\begin{aligned}\mathbb{E}[X_t] &= \mathbb{E}[(1-t)X_0 + tX_1] \\ &= (1-t)\mathbb{E}[X_0] + t\mathbb{E}[X_1] \\ &= (1-t)\mu_0 + t\mu_1\end{aligned}$$

Now, we know $Cov(X_t)$ can be solved using linear combinations therefore it can be derived as $Cov(X_t) = (1-t)^2\Sigma_0 + t^2\Sigma_1$. Then I have to solve for $Cov(X_1, X_t)$.

$$\begin{aligned}Cov(X_1, X_t) &= (1-t)Cov(X_1, X_0) + tCov(X_1, X_1) \\ &= (1-t)0 + t\Sigma_1 \\ &= t\Sigma_1\end{aligned}$$

And then I need to solve for $Cov(X_0, X_t)$

$$\begin{aligned}Cov(X_0, X_t) &= (1-t)Cov(X_0, X_0) + tCov(X_0, X_1) \\ &= (1-t)\Sigma_0 + t(0) \\ &= (1-t)\Sigma_0\end{aligned}$$

Now, I can substitute all these values into the original formulas. So we have

$$\begin{aligned}\mathbb{E}[X_1|X_t = x] &= \mathbb{E}[X_1] + \frac{Cov(X_1, X_t)}{Cov(X_t)}(x - ((1-t)\mu_0 + t\mu_1)) \\ &= \mu_1 + \frac{t\Sigma_1}{(1-t)^2\Sigma_0 + t^2\Sigma_1}(x - ((1-t)\mu_0 + t\mu_1))\end{aligned}$$

$$\begin{aligned}\mathbb{E}[X_0|X_t = x] &= \mathbb{E}[X_0] + \frac{Cov(X_0, X_t)}{Cov(X_t)}(x - \mathbb{E}[X_t]) \\ &= \mu_0 + \frac{(1-t)\Sigma_0}{(1-t)^2\Sigma_0 + t^2\Sigma_1}(x - ((1-t)\mu_0 + t\mu_1))\end{aligned}$$

$$\begin{aligned}
\mathbb{E}[X_1 - X_0 | X_t = x] &= \mathbb{E}[X_1 - X_0] + \frac{\text{Cov}(X_1 - X_0, X_t)}{\text{Cov}(X_t)}(x - \mathbb{E}[X_t]) \\
&= \mu_1 - \mu_0 + \frac{\text{Cov}(X_1 - X_0, X_t)}{\text{Cov}(X_t)}(x - \mathbb{E}[X_t]) \\
2. \quad &= \mu_1 - \mu_0 + \frac{\text{Cov}(X_1 - X_0, (1-t)X_0 + tX_1)}{\text{Cov}(X_t)}(x - \mathbb{E}[X_t]) \\
&= \mu_1 - \mu_0 + \frac{(1-t)\text{Cov}(X_1, X_0) + t\text{Cov}(X_1, X_1) - ((1-t)\text{Cov}(X_0, X_0) + t\text{Cov}(X_0, X_1))}{\text{Cov}(X_t)}(x - \mathbb{E}[X_t]) \\
&= \mu_1 - \mu_0 + \frac{t\Sigma_1 - (1-t)\Sigma_0}{(1-t)^2\Sigma_0 + t^2\Sigma_1}(x - ((1-t)\mu_0 + t\mu_1))
\end{aligned}$$

Then we want the velocity in affine form.

$$v^*(x, t) = \mu_1 - \mu_0 + \frac{t\Sigma_1 - (1-t)\Sigma_0}{(1-t)^2\Sigma_0 + t^2\Sigma_1}(x - ((1-t)\mu_0 + t\mu_1))$$

We can define $A_t = \frac{t\Sigma_1 - (1-t)\Sigma_0}{(1-t)^2\Sigma_0 + t^2\Sigma_1}$ which will give us

$$v^*(x, t) = \mu_1 - \mu_0 + A_t(x) - A_t((1-t)\mu_0 + t\mu_1)$$

Then define $b_t = \mu_1 - \mu_0 - A_t((1-t)\mu_0 + t\mu_1)$ and this will give us

$$v^*(x, t) = A_t(x) + b_t$$

3. The velocity produces the following gaussian distribution

$\mathcal{N}((1-t)\mu_0 + t\mu_1, (1-t)^2 + t^2)$. With $t=0$, we can compute the gaussian as

$$\mathcal{N}((1-t)\mu_0 + t\mu_1, (1-t)^2 + t^2) = \mathcal{N}((1-0)\mu_0 + 0\mu_1, (1-0)^2 + 0^2) = \mathcal{N}(\mu_0, 1)$$

. With $t=1$, we can compute the gaussian as

$$\mathcal{N}((1-1)\mu_0 + 1\mu_1, (1-1)^2 + 1^2) = \mathcal{N}(0 + \mu_1, 1) = \mathcal{N}(\mu_1, 1)$$